Java as a Database Frontend:

- The front end is software which is used to design the screens,menus,reports etc through which user can interact with the system.
- The backend is software mainly referred as database management system ,used to store the data and organize it in accessible form.

Introduction to JDBC

- **JDBC** stands for **Java Database Connectivity.**
- It is a set of Java APIs used for executing SQL statements.
- This API consists of a set of classes and interfaces to enable programmers to write pure Java Database applications.
- The JDBC API includes two packages:**java.sql** and **javax.sql**.
- The JDBC library includes APIs for each of the tasks mentioned below that are commonly associated with database usage.
- Making a connection to a database.
- Creating SQL or MySQL statements.
- Executing SQL or MySQL queries in the database.
- Viewing & Modifying the resulting records.

ODBC

- ODBC stands for **O**pen **D**atabase **C**onnectivity.
- It is an API which is used to access different databases.
- An ODBC driver uses the Open Database Connectivity (ODBC) interface by Microsoft that allows applications to access data in database management systems (DBMS) using SQL as a standard for accessing the data.
- **Key Features and Benefits**
- **Interoperability:** Allows a single application to connect to multiple database types seamlessly.
- **Platform Support:** Primarily designed for Windows, but also supported on Linux, Unix, and macOS.

- **Flexibility:** Supports both DSN (configured settings) and DSN-less (connection strings) connections.
- **Independence:** Applications are not locked into specific database vendors.
- ODBC is often compared to JDBC (Java Database Connectivity), though JDBC is specifically for Java applications, whereas ODBC is widely used for C/C++ and other applications

Two-Tier Architecture of JDBC

- The **two-tier architecture of JDBC** is a simple client-server model where the **client application directly communicates with the database** using JDBC.
- It consists of **two layers**:
- **Client Tier (Application)**
- **Database Tier (Database Server)**
- **1. Client Tier**
- Contains **user interface + business logic**
- Directly uses **JDBC API** to connect to database
- Sends SQL queries and receives results
- Example:
- Java Swing application
- Java console program

- **2. Database Tier**
- Database server that stores data
- Executes SQL queries sent by client
- Examples:MySQL ,Oracle ,SQL Server
- **Working of Two-Tier Architecture**
- User interacts with client application
- Client loads JDBC driver
- Client establishes connection with database
- SQL query is sent to database
- Database processes query
- Result is returned to client
- Client displays output

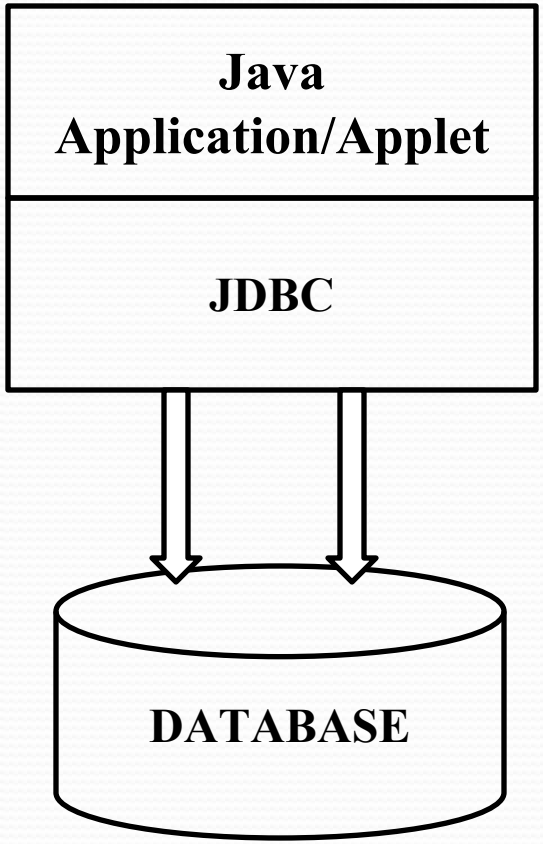


Fig. 2-Tier architecture

Three-tier architecture of JDBC

- The **three-tier architecture of JDBC** is a design model where the application is divided into **three separate layers**, improving scalability, security, and maintainability.
- It divides the system into:
- **Client Tier (Presentation Layer)**
- **Middle Tier (Application/Business Logic Layer)**
- **Database Tier (Data Layer)**

- **1. Client Tier (Presentation Layer)**
- This is the **user interface**
- Accepts input from user and displays output
- Example: Web browser, GUI application
- **2. Middle Tier (Application Layer)**
- Contains **business logic**
- Uses **JDBC API** to interact with database
- Processes user requests and sends results
- Responsibilities:
- Validations
- Processing logic
- Database communication
- **3. Database Tier (Data Layer)**
- Contains the **database server**
- Stores and manages data

Examples of databases: MySQL ,Oracle ,SQL Server

- **Working of JDBC in Three-Tier**
- User sends request from client
- Request goes to middle tier
- Middle tier uses JDBC to connect database
- Query is executed in database
- Result is returned to middle tier
- Middle tier sends response to client

Types of JDBC Drivers:

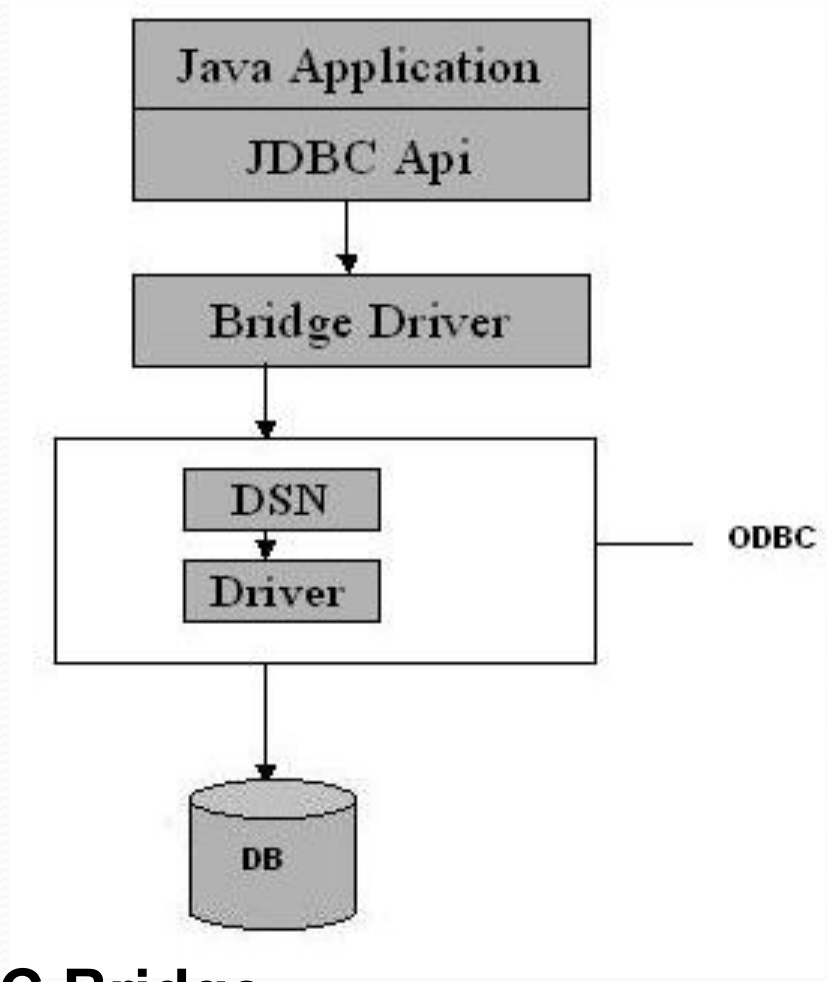
- **Type 1 Driver:**JDBC-ODBC Bridge driver
- **Type 2 Driver:**Native API/partly java driver
- **Type 3 Driver:**All java/Net-protocol driver
- **Type 4 Driver:**Native Protocol/all-java driver

Type1:JDBC-ODBC bridge driver:-

- **Type 1 Driver** is known as the **JDBC–ODBC Bridge Driver**. It is used to connect a Java application to a database through **ODBC drivers**.
- It acts as a **bridge** between JDBC and ODBC.
- **How It Works:**
- Java program sends request using **JDBC API**
- JDBC-ODBC bridge converts it into **ODBC calls**
- ODBC driver communicates with the **database**
- Result is returned back to Java program
- **Advantages**
- Easy to use for beginners
- No need for database-specific JDBC driver
- Can connect to any database supporting ODBC

Disadvantages

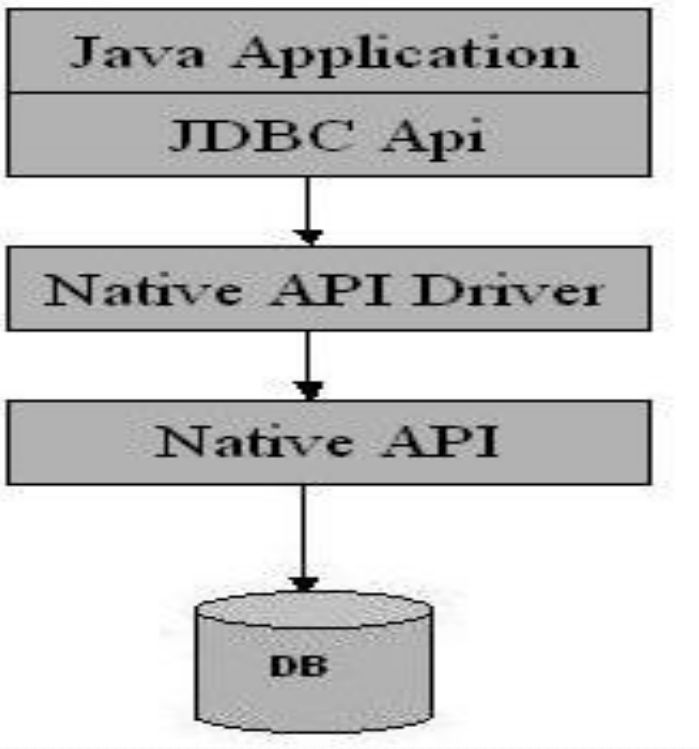
- **Slow performance** (two-step conversion: JDBC → ODBC → DB)
- **Platform dependent**
- Requires ODBC setup (DSN configuration)
- Not suitable for web or large applications
- **Removed in Java 8 and later**



Type 1: JDBC-ODBC Bridge

Type2:JDBC-Native API driver:-

- **Type 2 Driver** is called the **JDBC–Native API Driver**.
- It converts JDBC calls into **native database API calls** using **database-specific native libraries**.
- It is partly written in **Java + native language (C/C++)**
- **How It Works**
- Java program sends request using **JDBC API**
- Type 2 driver converts it into **native API calls**
- Native library communicates with database
- Result is returned back to Java program

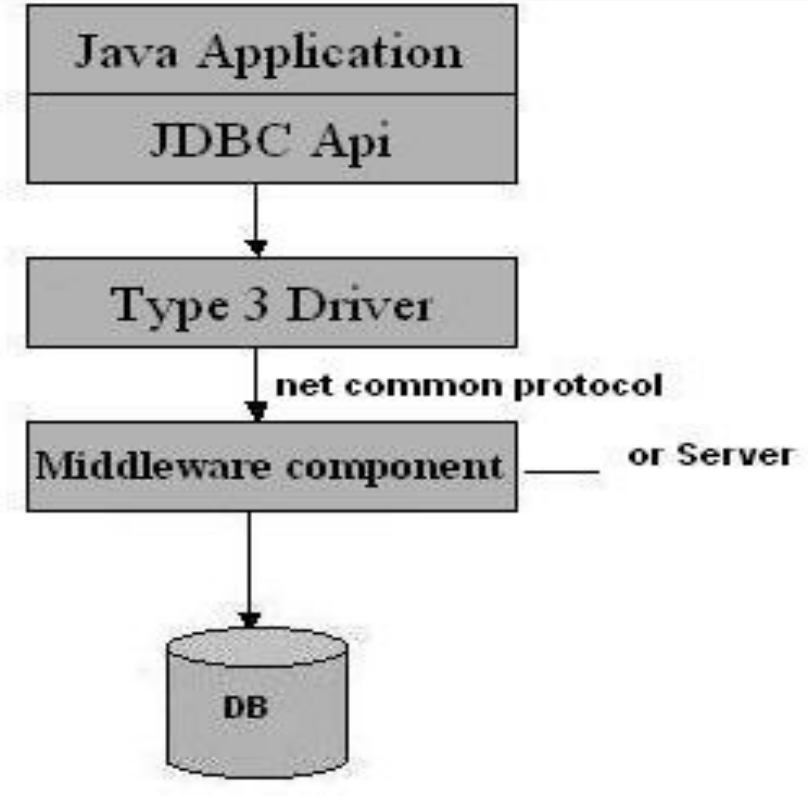


Type 2: JDBC Native api Driver

- **Advantages**
- Better performance than Type 1
- Direct communication with database
- Suitable for applications needing higher speed
- **Disadvantages**
- Platform dependent (native libraries required)
- Complex installation
- Not portable
- Not suitable for web applications
- Requires client-side configuration

Type3:JDBC-net pure Java driver:-

- **Type 3 Driver** is called the **JDBC–Net Pure Java Driver** or **Network Protocol Driver**.
- It converts JDBC calls into a **database-independent network protocol**, which is then sent to a **middleware server**.
- It is written entirely in **Java (pure Java driver)**
- **How It Works**
- Java application sends request using **JDBC API**
- Type 3 driver converts it into a **network protocol**
- Request is sent to **middleware server**
- Middleware converts it into database-specific calls
- Database processes request
- Result is sent back through middleware to Java application



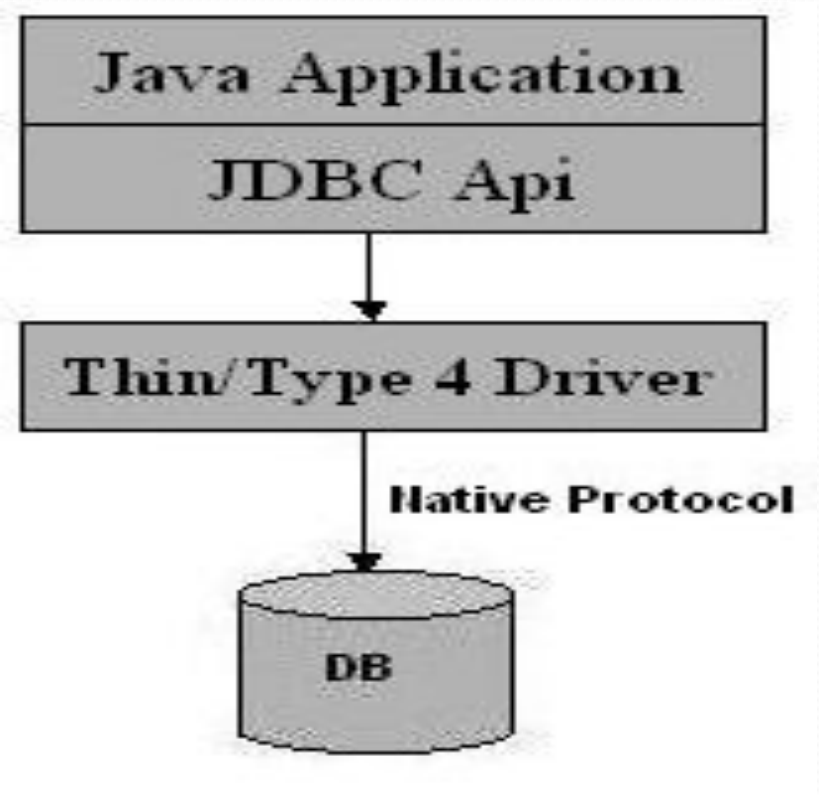
Type 3: All Java/ Net-Protocol Driver

- **Advantages**
- Platform independent (pure Java)
- No need to install database-specific drivers on client
- Supports multiple databases through middleware
- Better security (client does not directly access DB)
- Suitable for web and enterprise applications
- **Disadvantages**
- Requires middleware server
- More complex architecture
- Slightly slower than Type 4 (extra layer)
- Cost of maintaining middleware

Type4:100 % Native protocol pure Java driver:

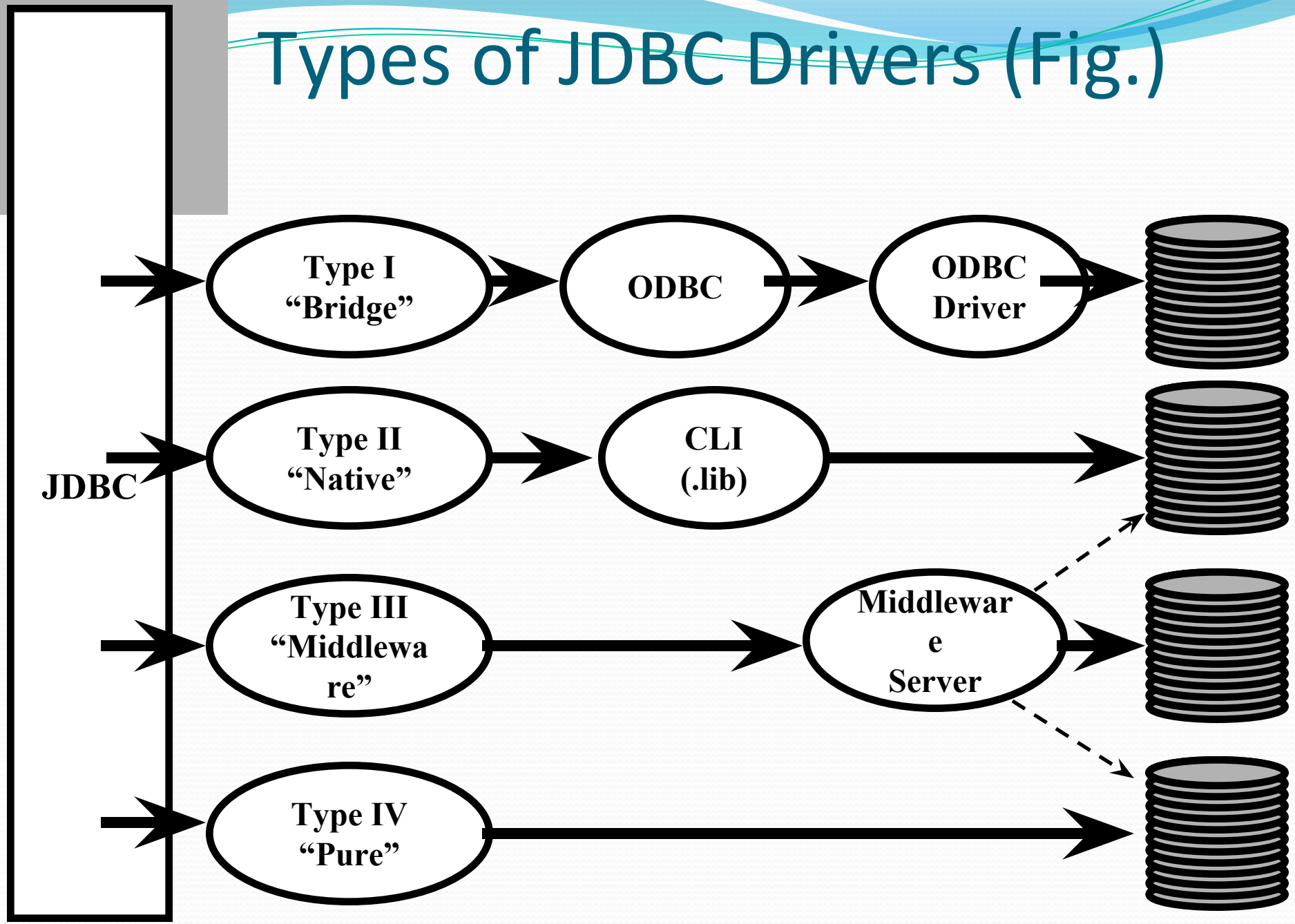
- **Type 4 Driver** is called the **Native Protocol Pure Java Driver**.
  It directly converts JDBC calls into the **database-specific protocol**.
- It is written entirely in **Java (pure Java driver)** and **does not require any middleware or native libraries**.
- **How It Works**
- Java application sends request using **JDBC API**
- Type 4 driver converts it directly into **database protocol**
- Database processes the request
- Result is returned to Java application
- No intermediate layer (like ODBC or middleware)

- **Advantages**
- High performance (fastest driver)
- Platform independent
- No external dependency
- Easy to deploy
- Suitable for web and enterprise applications
- **Disadvantages**
- Database-specific (different driver for each DB)
- Requires driver for each database



Type 4: Native-protocol/all-Java driver

Types of JDBC Drivers (Fig.)



CLASSES:-

- JDBC programming interface is found in "java.sql" and "javax.sql" packages.
- The interfaces and classes are provided by Java to establish connection to the database, query the database and also to perform some advance operation on the data tables.
- Mosts of the methods of these throw SQLException if an error occurs at runtime.
- Given below some classes and interfaces which support connectivity between interface and database :-

The DRIVER Interface

- A database driver is an object specific to a database that defines how statements are executed against that particular database.
- The **Driver** interface provide various methods for getting information about the current database driver as well as the **connect()** method, which creates a connection object that can be used to access the database.

  Connection connect(String jdbc_url, properties info)
- This method takes a **url** string to locate and access the database, along with username and password that are included in a container class called **Properties**.
- A JDBC-URL includes all the necessary information that a connection requires to connect to the database like host ,port,database name.

- A JDBC-URL has the following syntax:
  jdbc:<subprotocol>:<subname(*datasource*)>
- Here , each driver has its own subprotocol. **subprotocol** defines the type of driver which is vendor specific and each subprotocol has its own syntax for the source.
- **subname** provides the database identification which can be network encoded name .

For example: jdbc:odbc:employ

- **<subname>**can be further divided into
  jdbc:<subprotocol>:<//*host[:port]/database*>

For example:

jdbc:mysql://foo.nowhere.com:4333/account

Following table lists down popular JDBC driver names and database URL.

RDBMS	JDBC driver name	URL format
MySQL	com.mysql.jdbc.Driver	jdbc:mysql://hostname/ databaseName
ORACLE	oracle.jdbc.driver.OracleDriver	jdbc:oracle:thin:@hostname:port Number:databaseName
DB2	COM.ibm.db2.jdbc.net.DB2Driver	jdbc:db2:hostname:port Number/databaseName
Sybase	com.sybase.jdbc.SybDriver	jdbc:sybase:Tds:hostname: port Number/databaseName

The DriverManager Class:-

- The **DriverManager** class acts as an interface between user and drivers.
- It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver.
- The **DriverManager** class maintains a list of Driver classes that have registered themselves by calling the method DriverManager.registerDriver().
- It uses the following call to locate , load and link the named class:

  Class.forName(driver)

Sr .No	Method	Description
1	public static void registerDriver(Driver driver)	is used to register the given driver with DriverManager.
2	public static void deregisterDriver(Driver driver)	is used to deregister the given driver (drop the driver from the list) with DriverManager.
3	public static Connection getConnection(String url)	is used to establish the connection with the specified url.
4	public static Connection getConnection(String url,String userName,String password)	is used to establish the connection with the specified url, username and password.

CONNECTION INTERFACE

- The connection interface represent a session, i.e. a connection between java application and the database.
- The object of this interface can be used to create all of the various statement objects for executing SQL statements and stored procedures.
- It also has the ability to control changes made to the actual data stored through transactions.
- There are two ways for creating a **Connection** object.
- The first one uses the **getConnection ()** method of the DriverManager object.

- This method takes the JDBC-URL string as parameter and goes through the list of registered drivers and if the driver supports the subprotocol and subname, a connection object is returned.
- e.g:

Connection c = DriverManager.getConnection(“jdbc:oracle://oraserver:1521/staff”,”scott”, “tiger”);

- In the other method , instead of using **DriverManager** class, it is possible to get a connection directly from the **Driver** object.
- e.g :

  Connection c = driver.connect(“jdbc:oracle://oraserver:1521/staff”,”scott”, “tiger”);

Here,**driver** is an object of the Driver type.

- Commonly used methods of Connection interface:
- **1) public Statement createStatement():** creates a statement object that can be used to execute SQL queries.
- **2) public PreparedStatement prepareStatement(String sql):** creates a Preparedstatement object that can be used to execute dynamic SQL queries.
- **3) public void setAutoCommit(boolean status):** is used to set the commit status.By default it is true.
- **4) public void commit():** saves the changes made since the previous commit/rollback permanent.
- **5) public void rollback():** Drops all changes made since the previous commit/rollback.

void close()

- The **close()** method explicitly closes the connection and is useful if you have a limited number of available connections.
- The connection will be closed automatically when the application is terminated.

Steps for querying the database with jdbc

Step1 . **Getting Connected**

- register the driver
  -using the Driver and Driver Manager objects.
- Connect to the database
  -creates the connection object

step2. **Making a Query**

- Creates a statement
  -creates Statement, PreparedStatement or CallableStatement objects.
- Obtain the result
  -Creates result set object.

Step3. **Processing result**

- Step through the result
- Assign result to java variables.

Step 4. **Closing objects**

- Close all objects involved.

The Statement Interface

- The statement object is used to execute static and dynamic SQL statement.
- A static statement does not take any arguments to be completed.
- It can be SELECT, DELETE, UPDATE,INSERT and even a stored procedure call.
- A SELECT statement returns data from the database, where as others do not returns any data from the database.
- The statement object is created by the **createStatement ()** method of the connection object
- E.g.

  Statement s = c.createStatement( );
- where c is a Connection object

- The **Statement** interface has the following methods:

  ResultSet executeQuery(String sql)
- This methods execute the query and returns the result as an object of ResultSet.

  int executeUpdate(String sql)
- This method execute the DELETE, INSERT and UPDATE statement and returns the number of affected rows.

  Boolean execute(String sql)
- This method is used when an SQL statement like procedure call , returns multiple result sets.It returns true if the sql string returns a ResultSet object and false if it returns an integer.

The ResultSet Interface:-

- The object of this interface is used to get information from an SQL SELECT statement.
- The object of ResultSet maintains a cursor pointing to a row of a table. Initially, cursor points to before the first row.
- By default, ResultSet object can be moved forward only and it is not updatable.
- It is like multidimensional array having a finite set of rows and columns. Each of these columns contains some data value or null.

- The object of the ResultSet is generated by executing **executeQuery( )** method of the statement object by passing a valid SELECT statement in to it.
- e.g.

ResultSet r = s.executeQuery("SELECT name, age FROM stud WHERE city ="Anagar");

- Here, S is a statement object .

- Various getX() methods of ResultSet enable you to access the data contained in the columns of the result set.
- Here X is any primitive data type or String,etc.
- These methods are overloaded to take either the column name or the column number(as ordered in SELECT)as an argument.
- Some of these methods are:

int getInt(int column_no)

int getInt(String column_name)

double getDouble(int column_no)

double getDouble (String column_name)

String getString(int column_no)

String getString(String column_name)

boolean getBoolean(String column_name)

Date getDate(int column_no)

Commonly used methods of ResultSet interface

1) public boolean next()	is used to move the cursor to the one row next from the current position.
2) public boolean previous()	is used to move the cursor to the one row previous from the current position.
3) public boolean first()	is used to move the cursor to the first row in result set object.
4) public boolean last()	is used to move the cursor to the last row in result set object.
5) public boolean absolute(int row)	is used to move the cursor to the specified row number in the ResultSet object.
6) public boolean relative(int row)	is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative.

The PreparedStatement Interface

- If we want to execute the same SQL statement several times, we can create a PreparedStatement object
- at the point of creation of a PreparedStatement object the SQL code is sent to the DB and compiled. Subsequent executions may therefore be more efficient than normal statements.
- PreparedStatementscan take parameters
- This interface extends the Statement interface.
- It is used to execute dynamic SQL statements .

- **PreparedStatement** provides methods for setting the dynamic parameters for the SQL statements.
- To create a PreparedStatement object,**prepareStatement()** method of the Connection object is used.
- E.g:

  String s="select * from employee where age=?";

  PreparedStatement p=c.prepareStatement(s);
- Here,c is a Connection object and ? Specifies the parameter whose value is passed at a later time,using the methods of PreparedStatement interface.

- Some of the methods of this interface are:

Method	Description
public void setInt(int paramIndex, int value)	sets the integer value to the given parameter index.
public void setString(int paramIndex, String value)	sets the String value to the given parameter index.
public void setFloat(int paramIndex, float value)	sets the float value to the given parameter index.
public void setDouble(int paramIndex, double value)	sets the double value to the given parameter index.